

Maximum Sum Path in Two Sorted Arrays

Problem Statement

Given two **sorted arrays of distinct integers** $a[]$ and $b[]$, find the maximum sum path from the beginning of any array to the end of any array.

You can:

- Start from either array.
 - Move only from left $\rightarrow$ right.
 - Switch arrays **only at common elements**.
 - Count the common element only once.
-

Example 1

```
a = [2,3,7,10,12]
b = [1,5,7,8]
```

Output:

```
35
```

Path:

```
1  $\rightarrow$  5  $\rightarrow$  7  $\rightarrow$  10  $\rightarrow$  12
```

Sum:

```
1+5+7+10+12 = 35
```

Category

This problem belongs to:

- Arrays 
- Two Pointers 
- Greedy 

This is primarily a:

Two Pointers + Greedy on Sorted Arrays problem.

Why is this problem difficult?

Most array questions ask:

- maximum
- minimum
- count
- subarray

This question asks:

At certain points, I can switch paths.

That immediately makes our brain think:

Choices → Recursion/DP

But there is a hidden greedy observation.

Step 1: Understand the structure

Example:

```
A : 2 3 | 7 | 10 12
B : 1 5 | 7 | 8
```

Notice:

Before 7:

A = 2+3

B = 1+5

After 7:

A = 10+12

B = 8

The common elements divide the arrays into independent segments.

Key Observation

Between two common elements:

```
YOU CANNOT SWITCH.
```

Because switching is only allowed at common elements.

Therefore:

```
Between two common elements,  
you either travel entirely in A  
or entirely in B.
```

So we only need:

```
Maximum(segment sum in A,  
         segment sum in B)
```

This is the entire idea.

How to Think (Interview Thinking)

Whenever you see:

```
Two sorted arrays  
Switch allowed only at certain points  
Need maximum path/sum
```

Ask:

Question 1

Where can I switch?

Answer:

Only at common elements.

Question 2

Can I switch between two common elements?

Answer:

No.

Question 3

Then what matters between two common elements?

Answer:

Only the larger segment sum.

Question 4

Arrays are sorted?

Answer:

Use Two Pointers.

Brute Force Approach

At every common element:

Continue in current array
OR
Switch to other array

This becomes recursion.

Recursive State

```
f(i,j,currentArray)
```

Meaning:

```
Maximum path sum starting  
from current position.
```

Why Brute Force is Bad

Every common element creates:

```
2 choices.
```

So complexity becomes:

```
O(2^k)  
  
k = number of common elements
```

Very expensive.

Optimized Thinking

Instead of exploring both paths:

At every common element:

```
Take whichever path accumulated  
the larger sum.
```

Because after the common element:

```
Both paths merge.
```

The smaller path can never become useful again.

Algorithm

Maintain:

```
sum1 → current segment sum in A  
sum2 → current segment sum in B  
ans → final answer
```

Why Two Pointers?

Arrays are sorted.

If:

```
a[i] < b[j]
```

Then:

```
a[i] cannot appear later in B.
```

So we safely move:

```
i++
```

Similarly:

```
a[i] > b[j]
```

Move:

```
j++
```

When:

```
a[i] == b[j]
```

We found a switching point.

Code Building Step-by-Step

Step 1

```
i = 0
j = 0

sum1 = 0
sum2 = 0
ans = 0
```

Step 2

Traverse both arrays.

```
while i < n and j < m:
```

Case 1

```
a[i] < b[j]
```

This element belongs to current segment in A.

```
sum1 += a[i]
i += 1
```

Case 2

```
a[i] > b[j]
```

This element belongs to current segment in B.

```
sum2 += b[j]
j += 1
```

Case 3

```
a[i] == b[j]
```

Common element.

Choose larger segment.

```
ans += max(sum1, sum2)
```

Add common element once.

```
ans += a[i]
```

Reset segment sums.

```
sum1 = 0
sum2 = 0
```

Move both pointers.

```
i += 1
j += 1
```

Remaining Elements

One array may still have elements.

```
while i < n:
    sum1 += a[i]
    i += 1
```



```
while j < m:  
    sum2 += b[j]  
    j += 1
```

Final Segment

```
ans += max(sum1, sum2)
```

Complete Code

```
def maxPathSum(a, b):  
    n = len(a)  
    m = len(b)  
  
    i = 0  
    j = 0  
  
    sum1 = 0  
    sum2 = 0  
    ans = 0  
  
    while i < n and j < m:  
  
        if a[i] < b[j]:  
            sum1 += a[i]  
            i += 1  
  
        elif a[i] > b[j]:  
            sum2 += b[j]  
            j += 1  
  
        else:  
            ans += max(sum1, sum2)  
            ans += a[i]  
  
            sum1 = 0  
            sum2 = 0  
  
            i += 1  
            j += 1  
  
    while i < n:  
        sum1 += a[i]  
        i += 1  
  
    while j < m:
```

```
        sum2 += b[j]
        j += 1

    ans += max(sum1, sum2)

    return ans
```

Dry Run

```
A : 2 3 |7| 10 12
B : 1 5 |7| 8
```

Before 7:

```
sum1 = 5
sum2 = 6
```

At 7:

```
ans += max(5,6)
ans += 7

ans = 13
```

Reset:

```
sum1 = 0
sum2 = 0
```

After 7:

```
sum1 = 22
sum2 = 8
```

Finally:

```
ans += max(22,8)
```

```
ans = 35
```

Complexity Analysis

Time

```
O(n+m)
```

Space

```
O(1)
```

Pattern Recognition Template

When you see:

```
Two sorted arrays
+
Need maximum/minimum path
+
Can switch only at certain values
```

Think:

1. Switching points divide arrays into segments.
2. Between segments no switching possible.
3. Keep segment sums.
4. At switching point choose best segment.
5. Sorted arrays → Two Pointers.

Real World Mental Template

```
Train Route A ----- Junction ----- Junction
Train Route B ----- Junction ----- Junction
```

At every junction:

Take whichever route gave
more profit till now.

Identification Sheet

Keywords

two sorted arrays
common elements
switch allowed
maximum path
minimum path
merge point
intersection

Immediately think:

Two Pointers + Segment Sums + Greedy

Similar Problems

Intersection of Two Arrays

Leetcode 349

Pattern:

Two Sorted Arrays + Two Pointers

Common Elements in Three Sorted Arrays

Pattern:

Multiple Pointers

Merge Two Sorted Arrays

Pattern:

Two Pointers

Get the Maximum Score

Leetcode 1537

This is almost the exact same problem.

Print Common Elements in Two Arrays

Pattern:

Two Pointers

Union of Two Sorted Arrays

Pattern:

Two Pointers

Final Interview Template

IF:

1. Two sorted arrays
2. Need path/profit/score
3. Allowed to switch at common elements

THEN:

- Treat common elements as checkpoints.
- Compute segment sums.
- At each checkpoint take better segment.
- Use Two Pointers.
- $O(n+m)$.

Final Classification

Topic:

Arrays

Subtopic:

Two Pointers

Technique:

Greedy Segment Sum

Difficulty:

Medium